

從把程式寫出來，到讓一套系統真正運作

中文自傳 葉陽甫 | 國立花蓮高級中學

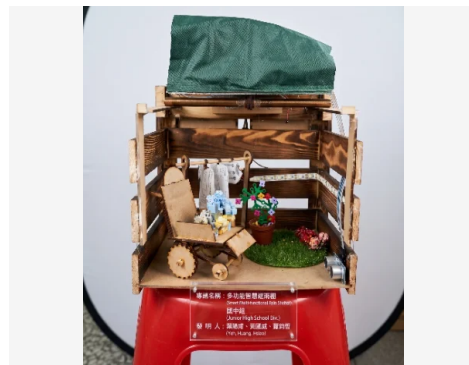
01 / 04



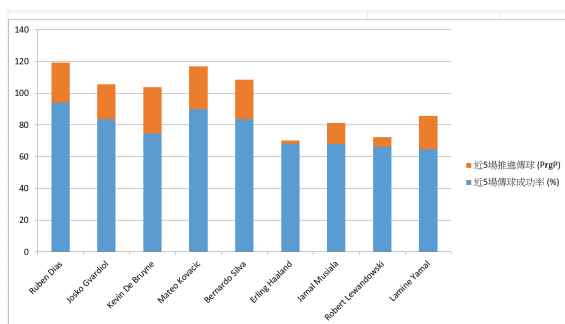
國中：第一次把程式接到實體世界

國中時，我和兩位同學做了一套多功能智慧遮雨棚，整合多種感測器與自動控制，App 控制介面由我獨立開發，作品獲得 IEYI 世界青少年發明展臺灣選拔賽銀牌。

這段經驗第一次讓我把程式、感測器與實體系統接在一起，也第一次看到程式在真實環境裡出錯。



國中發明展：第一次把程式、感測器與實體系統整合。



爬蟲抓下 FBref 數據後，整理成的球員戰術儀表板：近 5 場傳球成功率與推進傳球。

高一：IP 被封的那一次

高一時，我為了和朋友爭論球員近況，開始寫爬蟲抓 FBref (足球數據網站) 的球員資料。單一球員很快就成功，但連續送出大量請求後 IP 被封。我先換 VPN，結果新 IP 也被封，才回頭讀網站規範、理解 rate limiting，重新調整請求頻率與間隔。

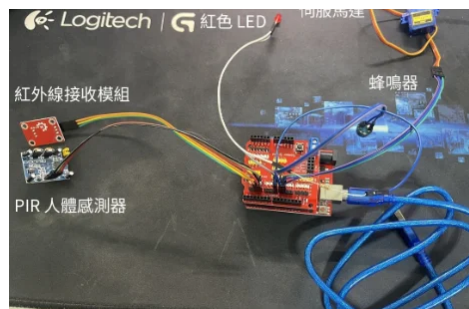
那次我才發現，真正卡住我的，不是程式寫不出來，而是我沒有理解正在使用的系統。

我不是從「會寫程式」開始，而是從一次次出錯中，慢慢學會理解系統。

高二：先拆開，再逐步整合

高二做 Arduino 防呆警報器時，模擬器一直讀不到紅外線訊號。我改用實體 Arduino，先單獨驗證接收器，再依序加入人體感測器、蜂鳴器與伺服馬達。

從那之後，我養成一個習慣：先拆開、逐一驗證，再整合。

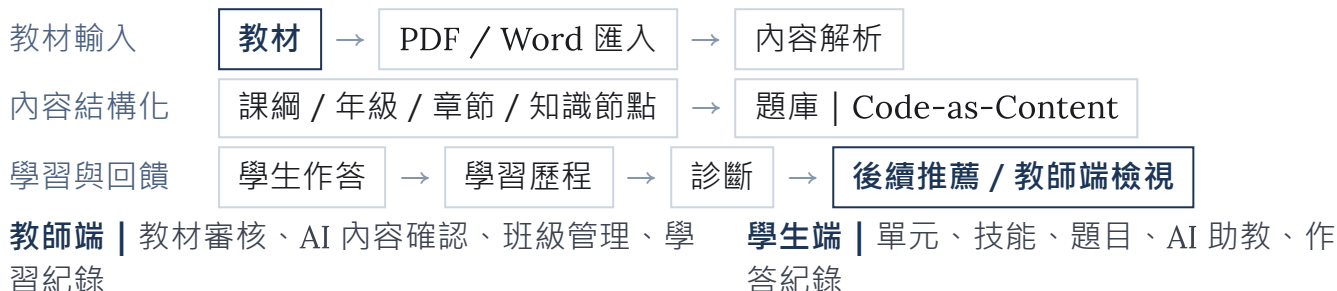


防呆警報器：逐一驗證每個模組後才整合。

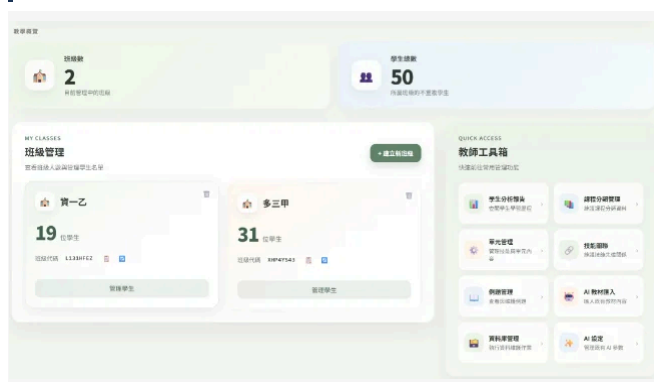
從單一功能，走向真正的大型 AI 學習系統

高二時，我主動向指導老師趙義雄老師詢問育秀盃的題目方向。想到父親在高職教數學時，常面對學生程度差異很大的問題，我和兩位同學因此開始做 AI 自適應學習系統。這是我第一次面對一套需要長期維運、真正讓教師與學生使用的系統，而不只是完成單一功能。

系統架構與我負責的資料主線 多角色 × 多模組 × 長資料鏈 × 長期維運



葉陽甫負責：從教材進入系統，到學生作答回傳的整條資料主線。



教師端：班級管理與 AI 教材匯入。

學生端：單元、技能與練習狀態。

延伸資料 | [系統實作影片：考卷診斷流程](#)

延伸資料 | [學生端展示：AI 即時救援](#)

大型專案難在哪裡

這不是單一功能，而是一條從教材匯入、內容解析、題庫生成、學生作答、學習紀錄、診斷到後續推薦的完整資料鏈。任何前段欄位、格式或模組變動，都可能讓後續流程一起失效。

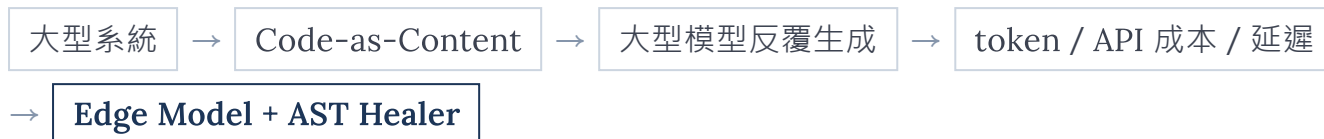
這套系統已經接受外部評審檢驗

系統後來獲得神通 AI 數位學院扶輪盃全國第一名，以及育秀盃高中職 AI 應用類金獎。對我而言，更重要的是：這套從真實教學問題出發、整合教材、題庫、作答與診斷流程的系統，第一次接受了團隊以外的外部檢驗。

真正改變我方向的，反而是得獎之後：系統繼續往下做，成本、延遲、錯誤與維運問題開始一個個出現。



大型專案開始真正運作後，問題才出現



Code-as-Content 的代價

題庫採用 Code-as-Content：把出題程式存進資料庫，由 Python 即時生成變題與答案。這讓一支程式可以反覆產生大量變題，但系統長期運作後，我開始看到另一個問題：每次都重新呼叫大型模型生成程式，token、API 成本與回應時間會持續累積；而同一數學單元中，許多程式結構與錯誤其實會反覆出現。

延伸資料 | [教材匯入與建立](#) ↗

已經解決過的問題，有沒有必要每一次都重新交給大型模型思考？



我把大型模型第一次完成的理解保留下來，讓後續相似問題不必每次都重新從頭處理。Edge Model 負責重複性的生成，Domain Function 與 Scaffolding 提供已整理好的工具與流程，AST Healer 則處理不需要重新理解題意、可由固定規則安全修復的局部錯誤。只有涉及答案、演算法，或必須重新理解題意時，才交回大型模型。

Edge Model 不必取代大型模型，只要讓大型模型不用一直重做已經解決過的事。

我的研究問題，是從大型專案的真實維運問題裡長出來的。

從漂亮分數，到知道什麼才算被證明

初版消融實驗在自創指標分數達到 97.41，我一度以為方法已經得到充分驗證。這份研究後來在東區科展獲得電腦與資訊學科優等，但沒有晉級全國。這讓我開始回頭檢查：問題到底出在哪裡？

到了旺宏科學獎，評審進一步追問：這個分數是在什麼條件下得到的？這套評分又能證明什麼？我重新拆解實驗流程，才發現測試題來自自己整理的規則，我的指標也是為這套系統設計的指標。97.41 沒有算錯，但它只能證明方法在當時的測試條件與評分框架下表現良好。

改變的，是我對「證明」的理解。

分數提升，和方法被充分證明，是兩件不同的事。

重新設計驗證

我後來發現，問題不只在測試資料，也在評分方式本身：不再問「分數有多高」，而是問「原本失敗的案例，有多少真的被安全救回來」。

改用公開會考試題

開發 / 評估分開

Healer 看不到正確答案

自訂分數 → 通過率

初版消融實驗 · MCRI
(自創指標 Math Code Reliability Index)

41.00 → 94.39 → **97.41**

基準 → 加入 Scaffolding → 再加入 AST Healer

Held-out / answer-blind 驗證
· safe rescue rate

2.63%

342 個原始失敗 → 介入 42 個 → 安全救回 9 個

延伸閱讀 | [旺宏科學獎完整成果報告 ↗](#)

為什麼是資訊工程，為什麼是清大資工

從硬體、爬蟲到 AI 學習系統與程式修復，我反覆追問的都是：系統為什麼會失敗、問題能不能被準確找出來，以及怎麼證明自己真的修好了。我希望把靠專案拼起來的知識，接回完整的資訊工程基礎。

我要補的 | 資料結構、離散數學、演算法、計算機組織、作業系統。

我想深入的 | 程式分析、生成可靠性、AI 系統設計、大型軟體系統。

我希望把目前靠專案累積的實作經驗，在清大接上完整的資訊工程基礎，進一步走向系統與程式分析研究。

APCS 程式識讀 **92** 分 · 第五級 / 程式實作 **135** 分 · 第二級

這個落差誠實反映我的位置：讀得懂、拆得開，但演算法實作仍需要正式訓練。

系統下一步將進入父親任教班級與學校 AI 實驗班，共 **49** 位學生。我想驗證的，不只是成績有沒有提升，而是變化是否真的由系統造成。

我在程式這一層已經學過一次：分數提升，不等於方法有效。我不想在系統真正進入教室之後，再犯一次同樣的錯。